

역기구학 Day1 보고서

로봇 21기 예비인턴 최윤서

1. 역기구학 개념

1.1 역기구학이란

로봇 또는 드론의 end-effector가 도달해야하는 목표 위치와 자세가 주어졌을 때 이를 실현하기 위한 각 관절의 값을 계산하는 과정

다시 말해, 직교 좌표 공간에서 정의된 목표를 관절 공간으로 변환하는 문제
(end-effect 주어짐 -> 관절 각도 계산 -> end-effector까지 도달)

1.2 정기구학 vs 역기구학

역기구학과 반대로 각 관절각도가 주어지면 좌표 변환을 통해 end-effector의 좌표 계산
(관절 각도 주어짐 -> end-effector 좌표 계산)

1.3 용어

1) Joint

로봇의 링크들을 서로 연결하여 회전이나 직선운동을 가능하게 만드는 핵심 구성 요소
각 조인트는 보통 하나의 자유도를 가지며, 조인트의 개수와 조합에 따라 로봇이 움직일 수 있는 공간과 방향이 결정됨.

회전 관절은 축을 중심으로 회전 운동을 하는 관절, 사람의 팔꿈치나 어깨와 유사
직선 관절은 미끄러지듯 직선 방향으로 이동하는 관절

2) End-effector

로봇의 팔 가장 끝 부분에 부착, 작업 대상과 직접 상호작용하는 모든 장치
정확한 형태는 로봇의 용도에 따라 다 다름
그리퍼, 공구형 엔드 이펙터 등이 대표적임

3) COM(Centor Of Mass)

물체의 전체 질량에서 중심이 되는 특정 지점
물체 내에서 정확한 밸런싱 액션을 수행할 수 있는 지점
다양하게 퍼진 질량에서 그 중 평균 위치를 하나의 가하학적 점으로 나타낸 것
질량 중심과는 다른 개념(질량 중심: 물체의 질량 분포만을 고려/무게중심: 중력이 일정하게 작용하는 공간에서 중력의 합력이 모이는 지점)

4) ZMP(Zero Moment Point)

로봇이 지면을 딛고 서거나 걸을 때, 발과 지면 사이의 접촉면에서 수평 방향의 모멘트가 0이 되는 지점

지지 다각형 내부에 위치하면 로봇은 넘어지지않고 균형 유지

(지지 다각형: 로봇의 발과 지면이 닿아있는 면적의 테두리 안쪽 공간)

5) LIPM(Linear Inverted Pendulum Model)

이족 보행 로봇의 보행 제어에서 복잡한 로봇의 움직임을 단순화하여 분석하고 제어

다체 역학을 가진 로봇을 질량이 없는 다리와 모든 질량이 모인 하나의 질점으로 단순화

2. 행렬

2.1. 행렬의 계산

1) 덧셈/뺄셈

일반적인 덧셈 뺄셈과 동일, 각 자리에 있는 값끼리 더해주거나 빼면 됨

2) 곱셈

조건: 앞선 행렬 A의 크기가 $m \times n$ 이고 뒤의 행렬 B의 크기가 $n \times k$ 일 때만 곱셈 AB가 가능함. 즉, A의 열의 개수와 B의 행의 개수가 일치해야함.

계산 방법: 앞 행렬의 i번째 행과 뒤 행렬의 j 번째 열에 있는 성분들을 차례로 곱한 뒤 모두 더한 값이 행렬의 (i,j)가 됨.

$$\begin{cases} ax + by + cz = p \\ dx + ey + fz = q \\ gx + hy + iz = r \end{cases} \Leftrightarrow \begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix} = \begin{pmatrix} p \\ q \\ r \end{pmatrix}$$

2.2 역행렬(Inverse)

어떤 행렬 A와 곱했을 때 곱셈에 대한 항등원인 단위행렬 E가 나오게 하는 행렬을 행렬 A의 역행렬이라고함. 기호는 A^{-1} 라고씀.

(이 때 E는 4.1에서 설명할 Identity Matrix이고 단위 벡터와 비슷한 단위 행렬임)

이차정사각행렬 $A = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ 에 대하여
 $ad - bc \neq 0$ 이면 $A^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$
 $ad - bc = 0$ 이면 행렬 A의 역행렬은 없다.

$A = [a \ b], [c \ d] / A^{-1} = [x \ y], [u \ v]$ 라고 놓고 $AA^{-1} = E$ 를 만족하는 A^{-1} 행렬을 a,b,c,d에 대해 표현해야함.

** $E = [1 \ 0], [0 \ 1]$

$$ax+bu = 1 \quad (1)$$

$$ay+bv = 0 \quad (2)$$

$$cx+du = 0 \quad (3)$$

$$cy+dv = 1 \quad (4)$$

$$(1) \times c - (3) \times a \text{를 해주면 연립방정식에 의해 } (bc-ad)u = c \text{가 나옴} \quad (5)$$

$$(1) \times d - (3) \times d \text{를 해주면 연립방정식에 의해 } (ad-bc)x = d \text{가 나옴} \quad (6)$$

$$(2) \times c = (4) \times a \text{를 해주면 연립방정식에 의해 } (bc-ad)v = -a \text{가 나옴} \quad (7)$$

$$(2) \times d - (4) \times b \text{를 해주면 연립방정식에 의해 } (ad-bc)y = -b \text{가 나옴} \quad (8)$$

이때 $(ad - bc) = 0$ 이라면 (5),(6),(7),(8)에서 $a=b=c=d=0$ 이고
 $a=b=c=d=0$ 이라면 (1)과 (4)에서 모순이 생김

$(ad-bc) \neq 0$ 이라면 (5),(6),(7),(8)에서 양변을 $(ad-bc)$ or $(bc-ad)$ 로 나눈다.
정리해서 x,y,u,v를 구하고 A^{-1} 에 대입하면 A의 역행렬을 구할 수 있음

2.3 전치 행렬(Transpose)

주어진 행렬의 행과 열을 서로 교환하여 얻은 행렬. 보통 A^T 라고 표현함
원래 행렬 A의 i행 j열에 있는 원소를 전치행렬의 j행 i열 원소로 옮김

만약 행과 열의 사이즈가 같다면 대각 선분을 기준으로 바뀌게 됨.
이를 대칭 행렬이라고함.

2.3.1 대칭 행렬의 성질

1) 임의의 행렬 $k \times n$ 과 $n \times m$ 행렬 A,B에 대해 다음과 같은 전치가 성립
 $[(AB)^T = A^T B^T]$

2) 전치행렬의 역행렬과 역행렬의 전치행렬은 항상 같다
 $[(A^{-1})^T = (A^T)^{-1}]$

$$\text{증명) } (A^{-1})^T A^T = (AA^{-1})^T = I^T = I$$

$$(A^T)^{-1} A^T = I$$

따라서 $(A^{-1})^T = (A^T)^{-1}$ 은 성립

2.4 행렬식(Determinent)

행렬식이란 정방행렬에 어떤 특정한 방법으로 하나의 수를 대응시키는 일종의 함수
즉, 행렬마다 각각 갖고있는 특정한 값을 나타내주는 것
(특정한 값: 행렬의 원소가 어떤 값이냐에 따라 다름)

$|A|$ 혹은 $\det(A)$ 라고 표기하며 determinant의 약자임

$$\det(A) = \sum_{j=1}^n a_{ij}(-1)^{i+j} M_{ij}$$

3. Linear Transformation(선형 변환)

3.1 Identity matrix

- 1) 어떠한 행렬에 identity matrix를 곱해도 그대로임
- 2) 즉, 아무 기능을 하지않음
- 3) 다른 행렬 변환의 시작이 됨 -> 1 값을 변환시키면서 다른 matrix 로직 탄생

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

3.2 Scaling(대각선)

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} a & 0 \\ 0 & d \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

- 1) Element a: x_1 이 x_2 에게 얼마나 영향을 미치는지 결정
- 2) Element d: y_1 이 y_2 에게 얼마나 영향을 미치는지 결정
- 3) 결론적으로 x축, y축 방향으로 얼마나 늘리거나 줄일지 결정함
- 4) $a > 0$ 이면 x 좌표 값이 0이 되고 $d > 0$ 이면 y 좌표값이 0이 됨

3.3 Shearing(비대각선)

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} 1 & b \\ c & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

- 1) Element b: 원래의 y좌표가 새로 변환된 x 좌표에게 얼마나 영향 줄지 결정
- 2) Element c: 원래의 x좌표가 새로 변환된 y좌표에게 얼마나 영향을 줄지 결정
- 3) 결론적으로 x,y 축을 얼마나 기울일지 결정함

3.4 Linear Transformation의 특징

- 1) 0은 항상 0에 대응된다.

행렬 안의 0이 있고 그에 해당하는 축의 좌표는 원점을 벗어날 방법이 없다
즉, 행렬 안에 0이 있다면 그 축의 좌표는 0에 대응된다

- 2) Linear Transformation은 항상 기함수(원점 대칭)이다.

어떤 점을 잡아서 그 점이 어떻게 움직이는지 보면, 그 점의 정확히 반대편(원점 대칭)은 반대 방향으로 움직인다

- 3) Linear Transform은 곱셈을 통해 연쇄적으로 이루어진다

만약 어떤 지점을 Scale하고 그 다음 Shear하고 그 다음 Rotate하고 싶다면 모든 matrix를 곱해야한다.

$$f_3(f_2(f_1(\mathbf{p}))) = \mathbf{A}_3\mathbf{A}_2\mathbf{A}_1\mathbf{p}$$

4. Rotation Matrix

4.1 Rotation Matrix의 특징

- 1) Inverse = Transpose

$$\begin{aligned}
 R(\theta)^{-1} &= \begin{bmatrix} \cos(-\theta) & -\sin(-\theta) \\ \sin(-\theta) & \cos(-\theta) \end{bmatrix} \\
 &= \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{bmatrix} \\
 &= \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}^T \\
 &= R(\theta)^T
 \end{aligned}$$

R 행렬을 역행렬해주면 세타 부호가 바뀌게 됨. $\cos(-a) = \cos(a)$ 이고 $\sin(-a) = -\sin(a)$ 를 적용해서 수식을 모두 풀어주면 R 행렬에서 행과 열을 바꾼 전치행렬의 결과가 나옴.

결론적으로 R행렬을 역행렬하던 전치행렬하던 결과가 똑같음.

전치행렬과 역행렬의 자세한 내용은 2.2와 2.3에 있습니다.

2) Determinent = 1

$$\begin{aligned}
 \det \begin{pmatrix} a & b \\ c & d \end{pmatrix} &= ad - bc \\
 \det(R(\theta)) &= \det \begin{pmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{pmatrix} \\
 &= (\cos(\theta) \cos(\theta)) - (-\sin(\theta) \sin(\theta)) \\
 &= \cos^2(\theta) + \sin^2(\theta) \\
 &= 1
 \end{aligned}$$

세타 값에 관계 없이 determinant는 항상 1임.

2.4에서 설명한 바와 같이 determinant는 정사각형 모양의 행렬을 대표하는 하나의 실수를 구하는 연산임. 수학적인 계산은 $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$ 가 있을 때 $ad-bc$ 로 계산한다.

Rotation Matrix에서 Determinent를 계산해보면 세타 값에 관계없이 항상 상수 1로 나오는 것을 위 사진에서 확인할 수 있음

3) Rotation x Rotation = Rotation

$$\begin{aligned}
 R(\alpha)R(\theta) &= \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \times \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \\
 &= \begin{bmatrix} \cos(\alpha) \cos(\theta) - \sin(\alpha) \sin(\theta) & -\cos(\alpha) \sin(\theta) - \sin(\alpha) \cos(\theta) \\ \sin(\alpha) \cos(\theta) + \cos(\alpha) \sin(\theta) & \sin(\alpha) \sin(\theta) + \cos(\alpha) \cos(\theta) \end{bmatrix} \\
 &= \begin{bmatrix} \cos(\alpha + \theta) & -\sin(\alpha + \theta) \\ \sin(\alpha + \theta) & \cos(\alpha + \theta) \end{bmatrix} \\
 &= R(\alpha + \theta)
 \end{aligned}$$

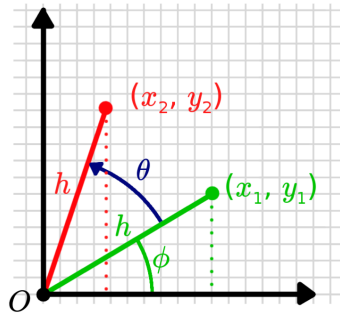
서로 다른 Rotation 2개를 곱하면 항상 또 다른 Rotation을 얻을 수 있음
 새로 얻은 Rotation은 다른 Rotation들과 동일한 성질을 가짐
 즉, 새로운 Rotation이 좌표계에 하나 더 생성되는 것

4.3 2D Rotation Matrix

2D 행렬 변환 공식

$$\begin{bmatrix} x_2 \\ y_2 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \end{bmatrix}$$

Def)



(오메가 기호는 a로 대체해서 설명, h = 1이라고 가정)

$$x_1 = \cos(a), y_1 = \sin(a), x_2 = \cos(\theta + a), y_2 = \sin(\theta + a)$$

삼각함수 덧셈공식에 의해 x2와 y2를 다음과 같이 작성할 수 있다.

$$x_2 = \cos(a)\cos\theta - \sin(a)\sin\theta \quad y_2 = \sin(a)\cos\theta + \cos(a)\sin\theta$$

이 때 $\cos(a) = x_1$ 이고 $\sin(a) = y_1$ 이므로 정리하면

$$x_2 = x_1\cos\theta - y_1\sin\theta \quad y_2 = y_1\cos\theta + x_1\sin\theta \quad \text{라고 쓸 수 있다.}$$

이를 보기 쉽게 행렬의 곱셈을 이용하여 표현하면 위와 같은 2D 행렬 변환 공식으로 표현할 수 있다. 행렬의 곱셈 순서는 2.1에 자세히 나와있다.

4.3 3D Rotation Matrix

$$\begin{bmatrix} z_{new} \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

2D Rotation Matrix에서 위 사진과 같은 z축을 추가해주면 3D 행렬 변환 공식은 다음과 같이 나타낼 수 있다.

$$\begin{bmatrix} x_{new} \\ y_{new} \\ z_{new} \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

5. Translation Matrix(이동 행렬 변환)

도형이나 점을 모양과 크기를 유지한 채 일정한 방향과 거리만큼 밀어서 옮기는
평행이동을 행렬의 곱셈으로 표현하기 위한 특수 형태의 행렬

(x,y) 라는 기저 벡터가 있을 때 이 벡터를 x축으로 a만큼, y축으로 b만큼
움직이고 싶다면 (x+a, y+b)로 표현할 수 있다.

그러나, 단순히 좌표를 평행 이동시키는 것처럼 좌표에 변화 값을 더하는 비선형 변환은
여러 변환을 행렬의 곱셈만으로 연결할 수 없다는 단점이 생긴다.

따라서, 6번에서 설명할 동차 행렬의 좌표 이동을 통해 이 문제점을 해결할 수 있다.

6. Transformation Matrix(동차 행렬)

동차 행렬이란 이동 행렬과 회전 행렬을 하나로 합친 행렬이다.
동차 좌표는 P(x,y)에서 1을 하나 추가해서 P(x,y,1)인 3개짜리 벡터로 만듦

5. Translation Matrix에서 발생한 문제점을 어떻게 해결할 수 있는가?
단순히 좌표에 이동할 값을 더하는 것이 아니라 아래 사진과 같이 Sx, Sy만큼
움직이고 싶다면 회전 행렬에 그 값을 추가해서 계산하면 된다.

$$\begin{bmatrix} x_1 + s_x \\ y_1 + s_y \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & s_x \\ 0 & 1 & s_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ y_1 \\ 1 \end{bmatrix}$$

회전과 이동이 모두 같은 동차 좌표 형식으로 표현되므로 두 행렬을 단순히
곱하기만 하면 회전 + 이동을 한면에 나타내는 행렬 하나를 만들 수 있다.

이때, 반드시 회전을 먼저, 이동을 나중에 적용해서 곱셈을 해야한다.
이동을 먼저 계산하게 되면 회전 기준점이 어긋나서 의도한 위치가 아닌 중심으로
회전하게 되기 때문이다.